INDEX

Programme: BSc Cybersecurity **Semester: V**

Course: Malware analysis

Sr. No.	Prac. No.	Particulars	Date	Teacher Signature
1	1	To install and configure all the software and tools required for the practical examinations.	20 August	
2	2	To analyze a file and prepare a report on it using static analysis, without executing it.	24 August	
3	3	To examine the structure of a Windows PE file and prepare a report on its headers and sections.	22 September	
4	4	To read and interpret a program's assembly code using IDA Free.	26 September	
5	5	To identify the Windows APIs used by a program and infer its behavior from them.	1 October	
6	6	Case Study 1: WannaCry Ransomware (2017)	6 October	
7	7	Case Study 2: Emotet Banking Trojan and Loader (2014–2021)	9 October	

PRACTICAL 1:
Installation of Malware Analysis Tools

1. Aim

To install and configure all the software and tools required for the practical examinations.

2. Background Information

Malware analysis tools help examine executable files, understand program structures, and analyse assembly instructions and Windows APIs. These tools are useful for identifying suspicious characteristics in files and understanding how programs work.

3. Software Required

1. Sysinternals Strings
2. Exeinfo PE
3. CFF Explorer
4. IDA Free
5. MSYS2

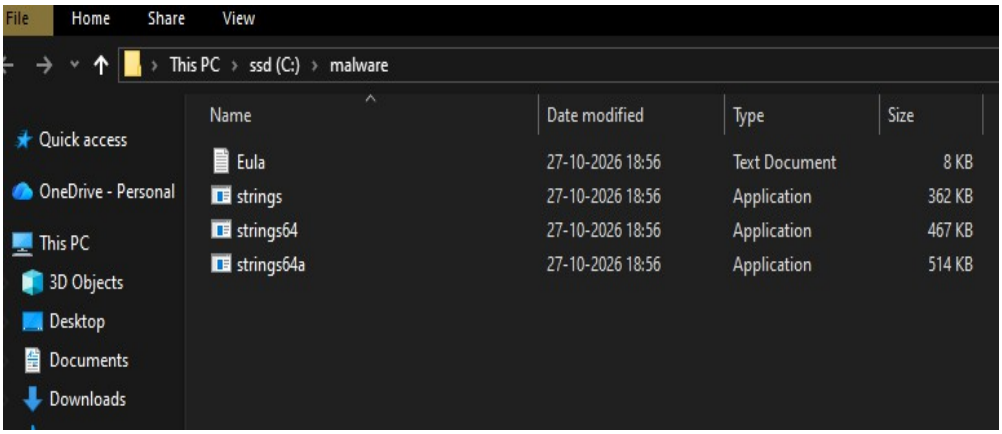
4. Download Links

1. **Sysinternals Strings:** https://learn.microsoft.com/en-us/sysinternals/downloads/strings
2. **Exeinfo PE:** https://sourceforge.net/projects/exeinfo-pe/files/
3. **CFF Explorer:** https://ntcore.com/explorer-suite/
4. **IDA Free:** https://hex-rays.com/ida-free/
5. **MSYS2:** https://www.msys2.org/

5. Installation Steps

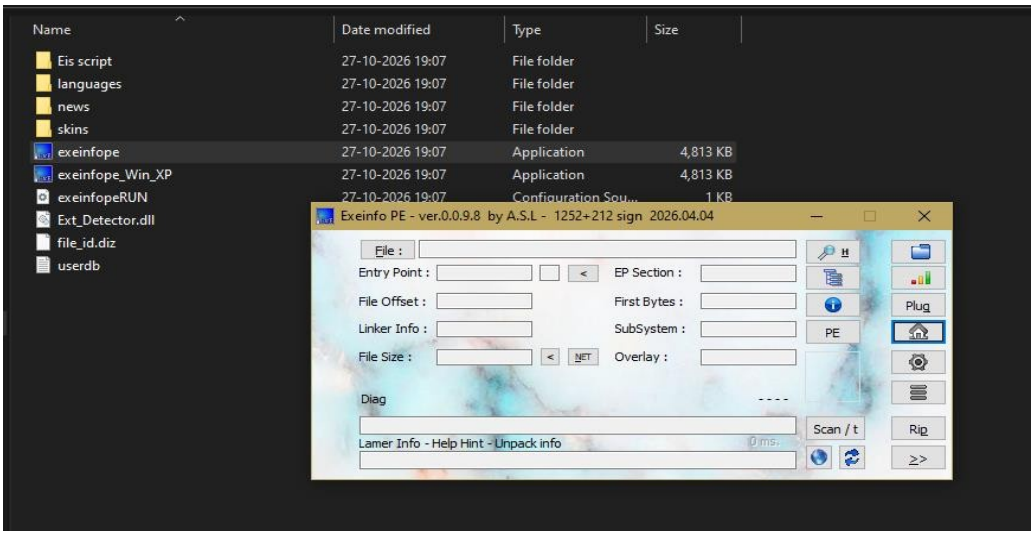
A. Sysinternals Strings

1. Open the official Microsoft Sysinternals website.
2. Download the Strings ZIP file.
3. Extract the downloaded ZIP file.
4. Locate strings.exein the extracted folder.



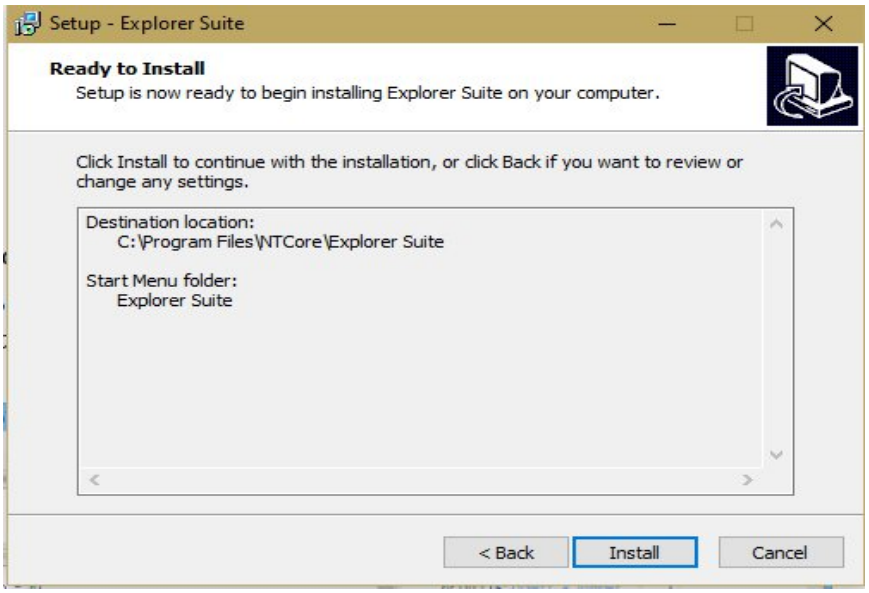
B. Exeinfo PE

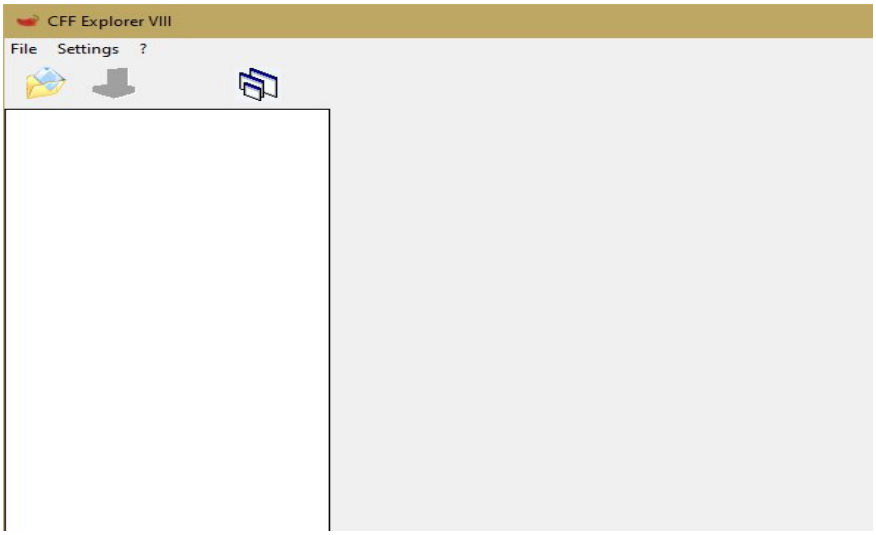
1. Open the Exeinfo PE download page.
2. Download the available software package.
3. Extract the files if the download is compressed.
4. Locate and open the Exeinfo PE application.



C. CFF Explorer

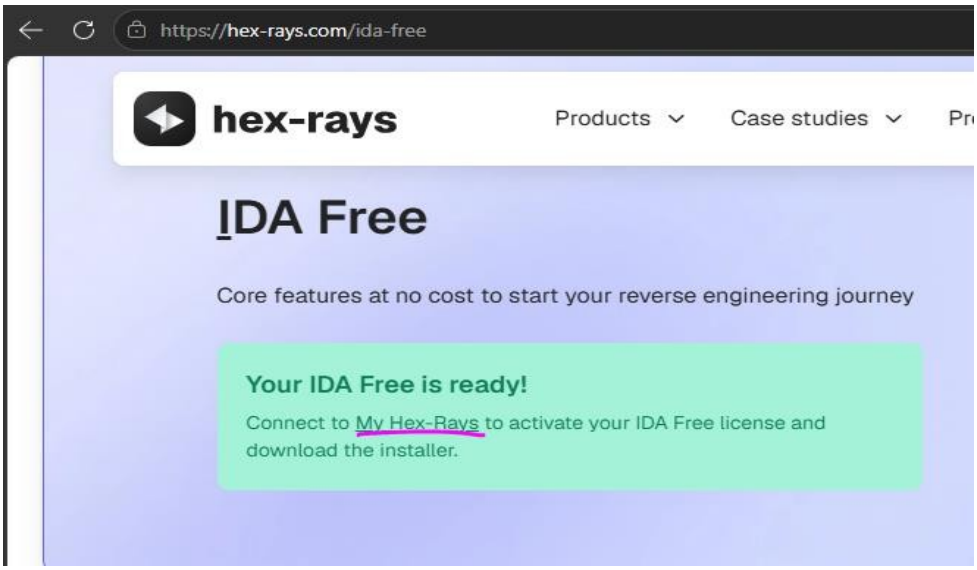
1. Open the official NTCore Explorer Suite website.
2. Download the Explorer Suite installer.
3. Run the downloaded installer.
4. Follow the installation instructions to complete the setup.
5. Open CFF Explorer.

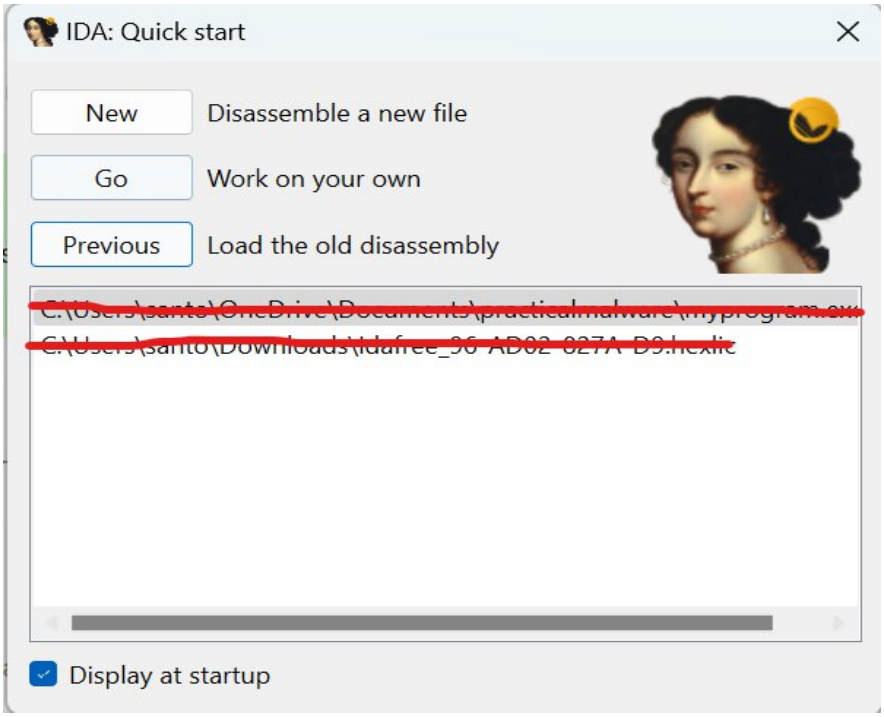




D. IDA Free

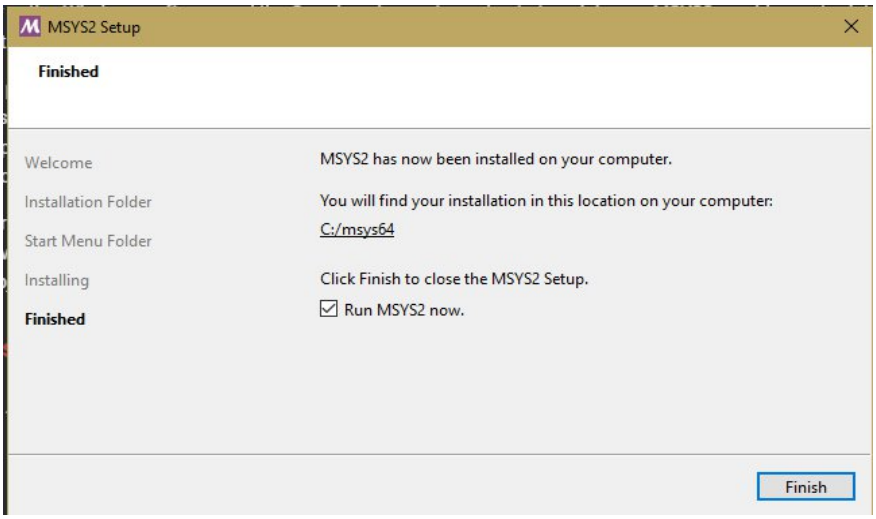
1. Open the official Hex-Rays website.
2. Select the IDA Free download option.
3. Complete the required registration or license steps.
4. Download the Windows installer.
5. Run the installer and follow the setup instructions.
6. Launch IDA Free after installation.

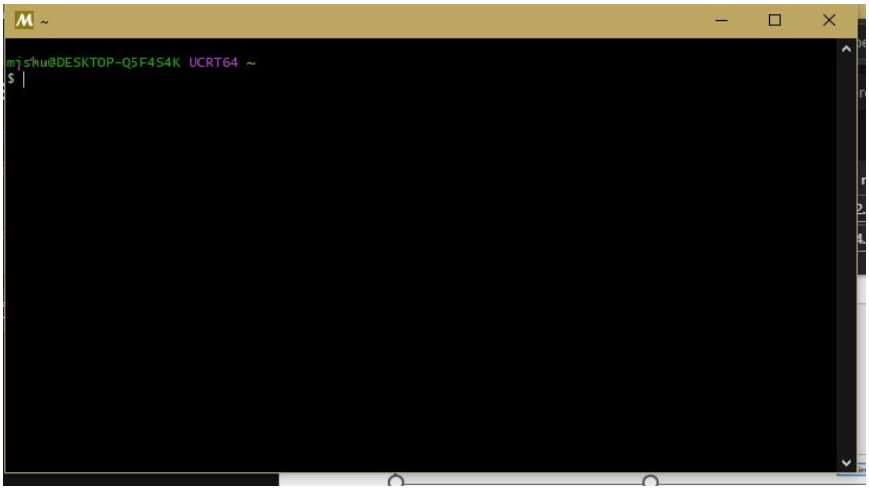




E. MSYS2

1. Open the official MSYS2 website.
2. Download the Windows installer.
3. Run the installer and follow the setup instructions.
4. Complete the installation and open MSYS2.





6. Result

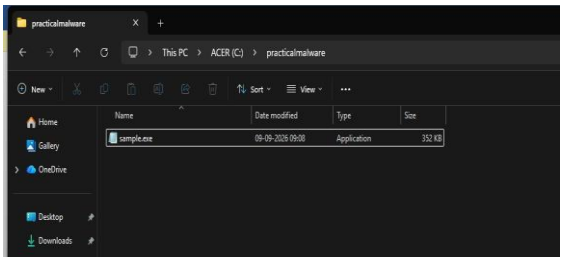
The software tools required for Malware Analysis were downloaded.

Practical 2

Aim: To analyse a file and prepare a report on it using static analysis, without executing.

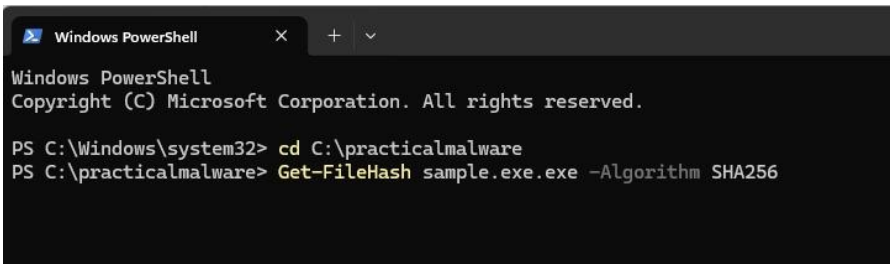
Step 0: Create Your Working Folder and Get a Test File

1. Create a folder named Malware_Analysis in your desired location.
2. Open C:\Windows\System32 in File Explorer and copy notepad.exe (Type: Application, not .mui).
3. Paste it into your practical folder and optionally rename it to sample.exe.



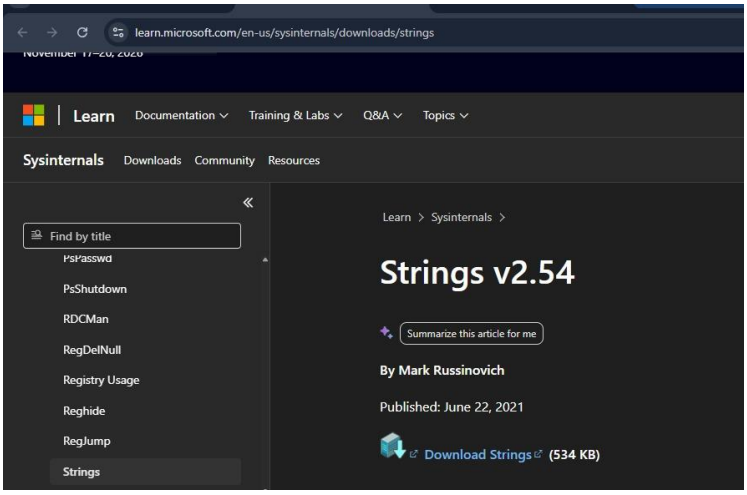
Step 1: Get the File's Hash (Fingerprint)

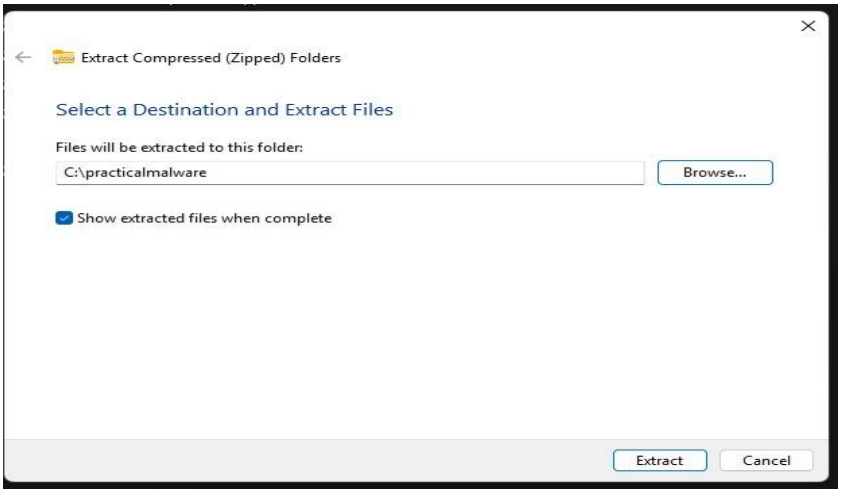
1. Open PowerShell, navigate to your folder, then run:
2. cd "C:\practicalmalware''
3. Get-FileHash sample.exe.exe -Algorithm SHA256
4. Copy the resulting hash somewhere safe — you'll need it for Step 4.

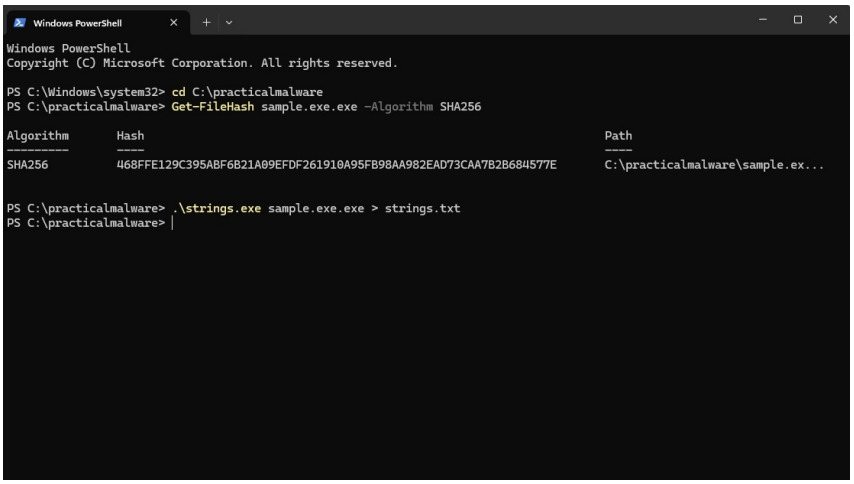


Step 2: Extract Readable Text (Strings)

1. Download Strings from https://learn.microsoft.com/en-us/sysinternals/downloads/strings and extract it into your practical folder.
2. Run .\strings.exe sample.exe > strings.txt in PowerShell.
3. Open the results using notepad strings.txt and check for URLs, file paths, suspicious commands, or unusual text.

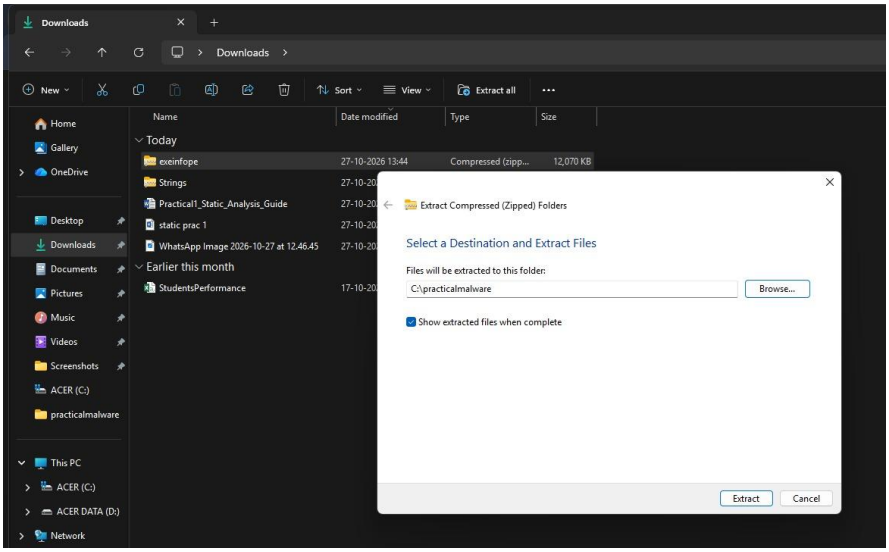


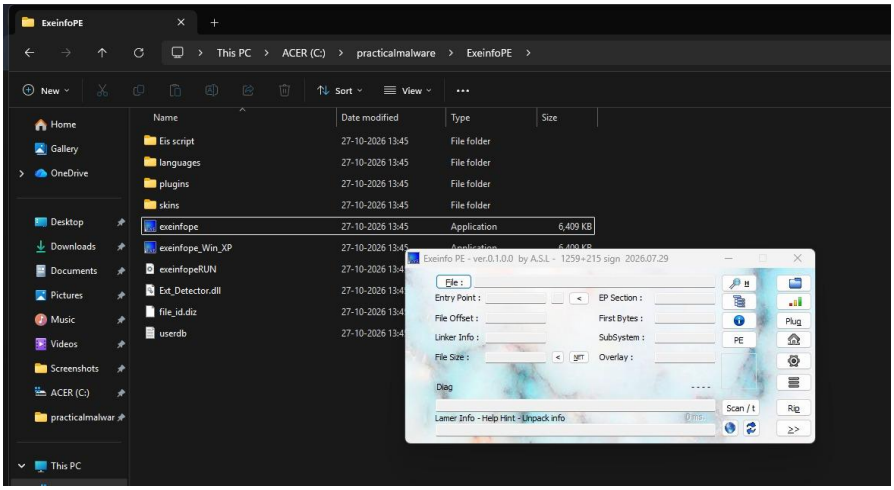


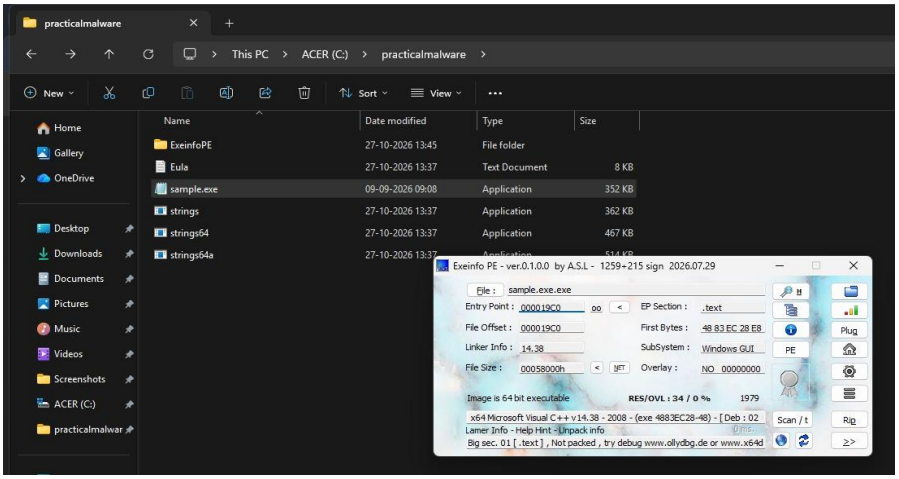


Step 3: Check Compiler & Packing (Exeinfo PE)

1. Download Exeinfo PE from https://github.com/ExeinfoASL/ASL and extract it into your practical folder.
2.Run exeinfope.exe and load your sample.exe file.
3.Check the results for the compiler information and whether the file is packed or unpacked.

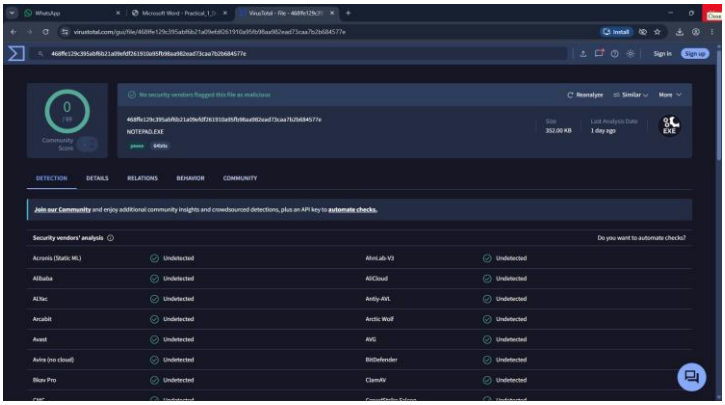






Step 4:

1. Visit VirusTotal and search using the SHA-256 hash obtained in Step 1.
2. Note the detection ratio (e.g., 0/71) to see how many antivirus engines flag the file.
3. For a safe test file, expect zero or very few detections, and disconnect the internet afterward if required.


No security vendors flagged this file as malicious
Reanalyze
Similar
More
NOTEPAD.EXE
Size
353.00 KB
Last Analysis Date
1 day ago
DETECTION
DETAILS
RELATIONS
BEHAVIOR
COMMUNITY
Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.
Security vendors' analysis
Do you want to automate checks?
Acronis (Static ML)
Undetected
AhnLab-V3
Undetected
Alibaba
Undetected
AliCloud
Undetected
ALYac
Undetected
Antiy-AVL
Undetected
Arcabit
Undetected
Arctic Wolf
Undetected
Avast
Undetected
AVG
Undetected
Avira (no cloud)
Undetected
BitDefender
Undetected
Bkav Pro
Undetected
ClamAV
Undetected

Report

Practical 1: Look at a File Without Running It — Report

Objective:

Check a file for warning signs without opening or running it, using its hash, hidden text, and an online checker.

File analyzed:

sample.exe (copy of notepad.exe)

1. File Fingerprint (Hash)

- Algorithm: SHA-256

Value: 1. 468FFE129C395ABF6B21A09EFDF261910A95FB98AA982EAD73CAA7B2B684577E

2. Strings Analysis

- Tool used: Sysinternals strings.exe

Findings: The strings.exe tool identified readable ASCII/Unicode strings in the executable, including PE file headers, section names such as .text, .rdata, .data, and .rsrc, and the message "This program cannot be run in DOS mode."

3. Packing Check

- Tool used: Exeinfo PE

Compiler identified: Microsoft Visual C++ 14.38 (2008–2026)

Packed? (Yes/No): No (not packed, according to Exeinfo PE)

4. VirusTotal Result

Detection ratio: 0/69

File type confirmed: PE executable (64-bit Windows EXE)

5. Conclusion

Based on the static-analysis results, sample.exe appears consistent with a legitimate Windows executable. VirusTotal reported 0/69 detections, and Exeinfo PE identified a 64-bit Microsoft Visual C++ executable that is not packed. However, these results do not prove the file is safe. The Strings output should be rerun directly on sample.exe to verify its contents before drawing a final conclusion.

Practical 3: To examine the structure of a windows PE file and prepare a report on its headers and sections

1. Objective

To open a Windows program (.exe) in a viewer and understand how it is built on the inside.

2. Tools and Sample Used

Item	Details
Tool (CFF Explorer / PE-bear)	CFF Explorer VII
Tool version	III
Sample file name	notepad.exe
Original location of sample	C:\Windows\System32\notepad.exe
File size	32-bit
File type (.exe / .dll)	.exe
Environment	Normal computer (no VM needed; file was only opened, never run)

3. Procedure

1. Opened the sample file in the tool.
2. Found the header details (file type, number of sections).
3. Listed all section names and identified the largest.
4. Recorded at least 10 imported functions and their source DLLs.
5. Checked the export list (only if the file is a .dll).
6. Guessed the program's purpose from the imported functions.

4. Observations

4.1 Header details

Field	Location in tool	Value observed
e_magic	Dos Header	5A4D
e_lfanew	Dos Header	000000F8
Signature	Nt Headers	00004550
Machine (32/64-bit)	File Header	8664
NumberOfSections	File Header	0008
Characteristics (EXE/DLL)	File Header	0022
Magic (PE32/PE32+)	Optional Header	020B
AddressOfEntryPoint	Optional Header	000019C0
Subsystem (GUI/Console)	Optional Header	0002

4.2 Sections

Section name	Purpose (your words)	Virtual Size	Raw Size	Biggest? (Y/N)
.text	Contains the program's executable machine code	000267E2	00027000	Y
fothk	Custom section; likely contains program-specific data/code	00001000	00001000	N
.rdata	Read-only data such as constants and strings	0000A6C8	0000B000	N
.data	Read/write global and static program data	00002740	00001000	N
.pdata	Exception/unwind information used for program execution	00001218	00002000	N
.didat	Delay-import information for DLL functions	000000F8	00001000	N
.rsrc	Program resources such as icons, dialogs, and version information	0001E1D0	0001F000	N

Largest section: .text ____ **Reason**: it has the largest virtual size, `000267E2`, so it occupies the most virtual memory among the listed sections.

4.3 Imported functions (minimum 10)

#	Function	From DLL	What it likely does
1	SetMapMode	GDI32.dll	Sets coordinate mapping
2	SetViewportExtEx	GDI32.dll	Sets viewport size
3	SetWindowExtEx	GDI32.dll	Sets window point
4	LPtoDP	GDI32.dll	Converts logical to device coordinates
5	SetBkMode	GDI32.dll	Sets background mode
6	GetTextMetricsW	GDI32.dll	Gets text / font matrices
7	TextOutW	GDI32.dll	Displays text
8	AbortDoc	GDI32.dll	Cancels printing
9	EndDoc	GDI32.dll	Ends printing
10	SetAbortProc	GDI32.dll	Sets print cancellation procedure
11	StartDocW	GDI32.dll	Starts a print document

5. Screenshots

Paste a screenshot in each box below and keep the caption.

[Paste screenshot: File Header / Optional Header]

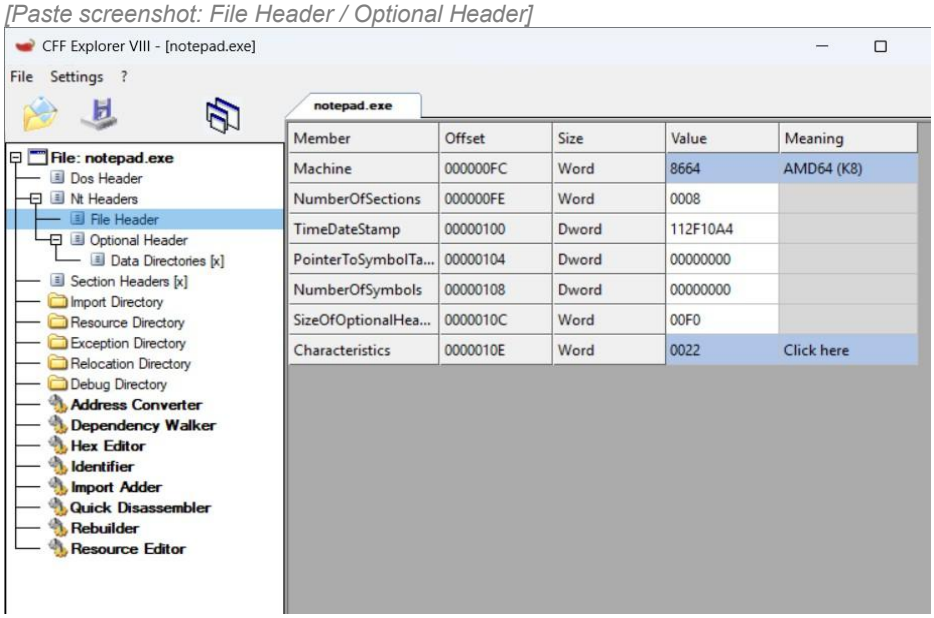
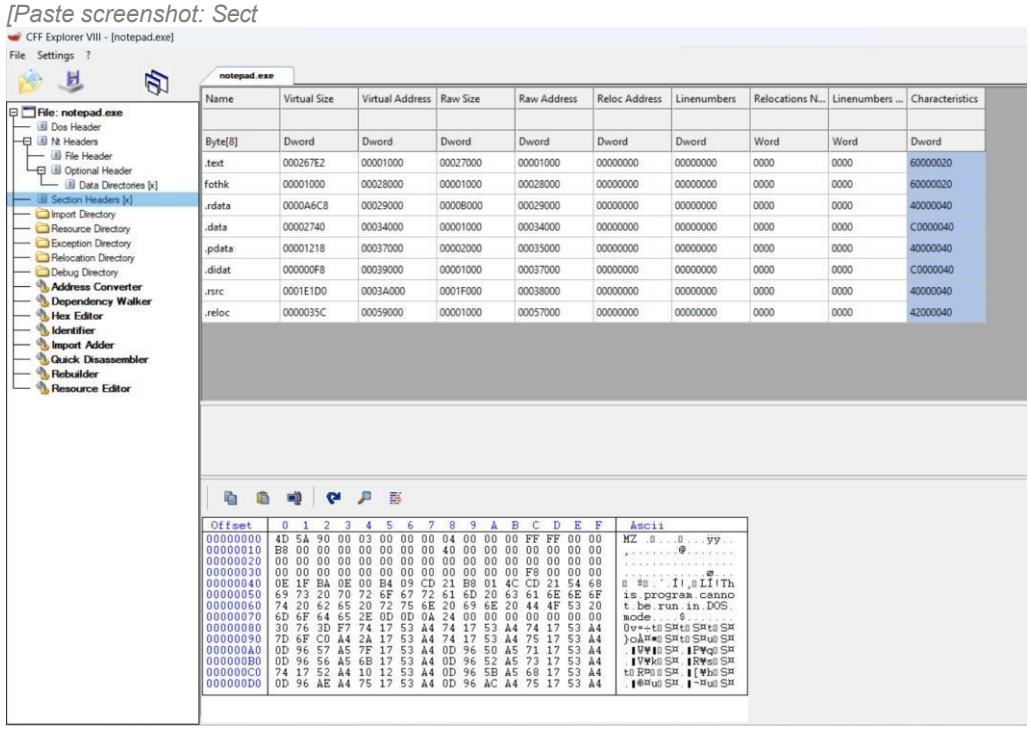


Figure 1: Header details

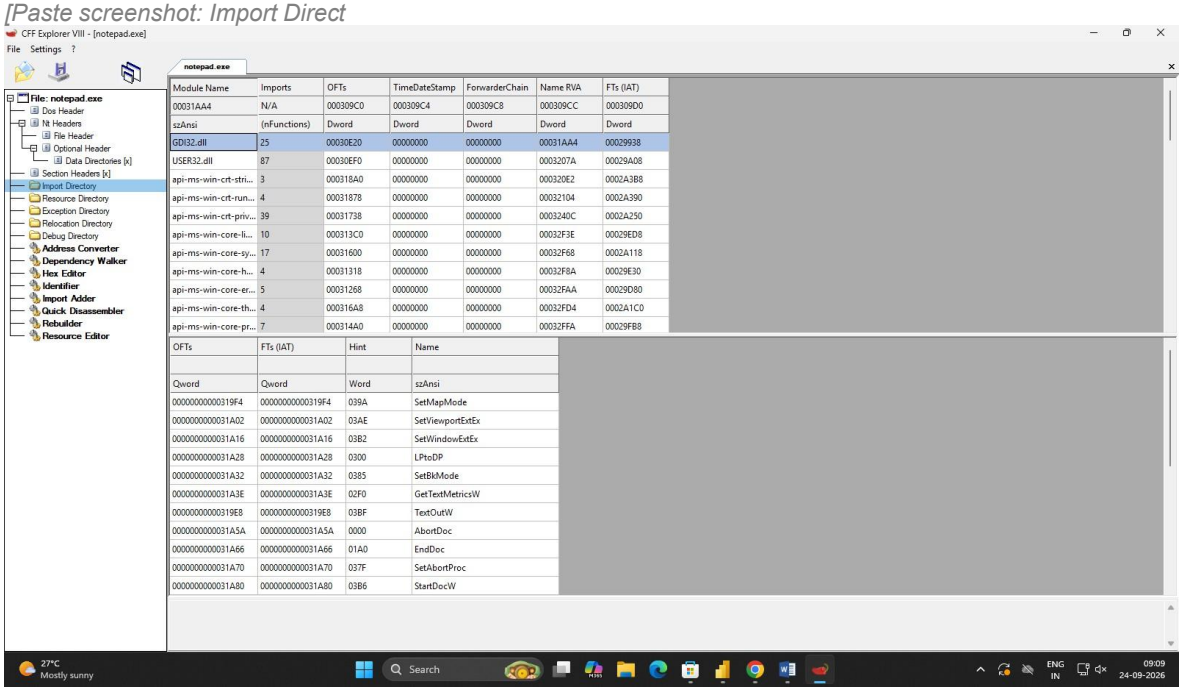
[Paste screenshot: Sect



ion Headers]

Figure 2: Section list

[Paste screenshot: Import Direct



ory]

Figure 3: Imported DLLs and functions

6. Analysis

Based on the imported functions, the program appears to be a **Windows GUI application** capable of interacting with the operating system. The imported APIs can be used for **file handling, creating and managing windows, memory management, and process/system operations**. If the imports include registry or network-related DLLs, those capabilities should also be mentioned, but the imported-function list is not visible in the screenshot, so those cannot be confirmed from the available information.

7. Conclusion.

The file is a **PE (Portable Executable) Windows executable**, and the section layout shown indicates a **32-bit executable**. It is a **GUI application**, consistent with `notepad.exe`, rather than a console program. The **`.text` section is the largest**, with a virtual size of `000267E2`. Based on the file name and PE structure, the program appears to be a **Windows text-editor application**.

Practical 4

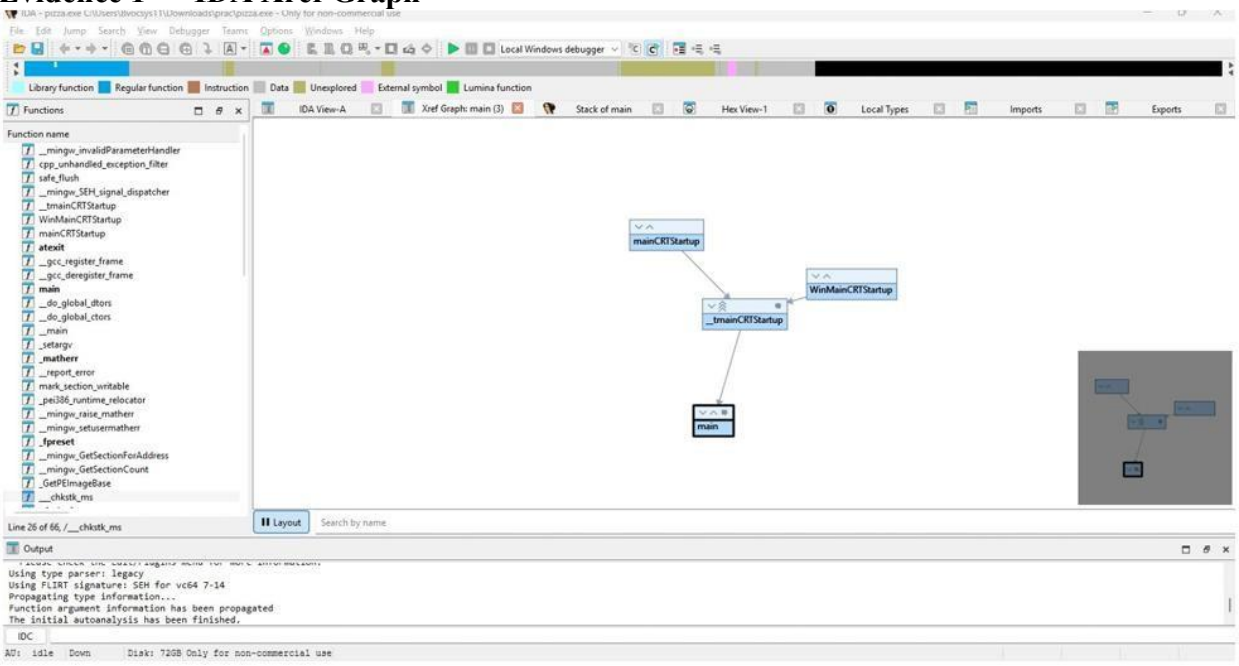
Aim: To read and interpret a program's assembly code using IDA Free.

Objective	To analyze a compiled C program using IDA Pro and identify the assembly instructions corresponding to a loop and an if-else condition, while tracing variables through their memory references.
Tools Used	IDA Pro, IDA View-A, Hex View, Xref Graph, Windows Local Debugger

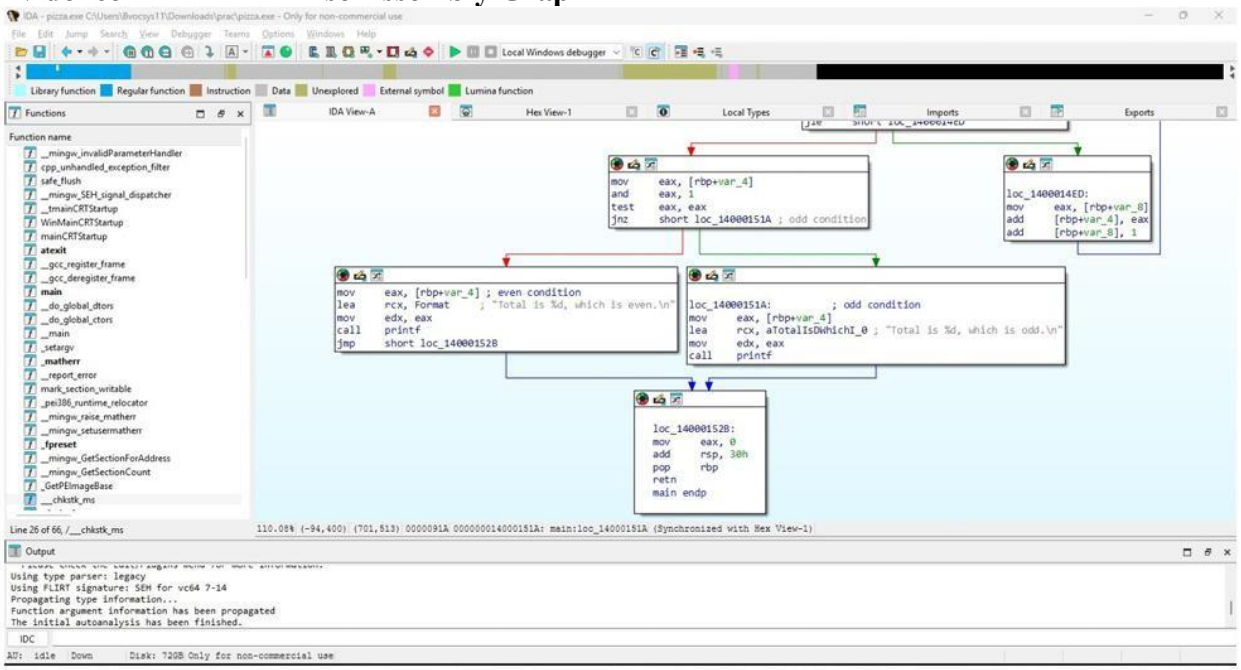
C Source Code (attach myprogram.c or paste below)	#include <stdio.h>
	int main() {
	int total = 0;
	int i = 0;
	for (i = 0; i <= 9; i++) {
	total += i;
	if (total % 2 == 0)
	printf("Total is %d, which is even.\n", total);
	else printf("Total is %d, which is odd.\n", total);
	}
	return 0;
	}

	Address: 0x1400014F7. The instructions cmp [rbp+var_8], 9 followed by jle short loc_1400014ED represent the for-loop condition (i <= 9). The loop body updates total and increments i using add [rbp+var_8], 1, then control returns to the loop-condition block.
If-Else — Address & Assembly Found	Around 0x1400014ED, mov eax, [rbp+var_4], and eax, 1, test eax, eax and jnz short loc_14000151A implement the parity test. A non-zero result branches to the odd case; otherwise execution continues to the even case.
Xrefs Findings (variable traced + where read/written)	var_4 represents total: initialized to 0, updated by add [rbp+var_4], eax, then read for the parity test and printf. var_8 represents i: initialized to 0, read by cmp [rbp+var_8], 9, and incremented with add [rbp+var_8], 1. The Xref Graph also shows the startup path reaching main.
Mapping Summary (C code ↔ Assembly, in your own words)	The C for loop is implemented with cmp and a conditional jle jump, followed by an increment of the loop variable. The if (total % 2 == 0) condition is implemented with and/test and jnz. The two control-flow branches call printf with separate even and odd format strings.
Attached Evidence Files (.idb/.i64, .c, screenshots)	pizza.exe / IDA Pro analysis, C source code, IDA Xref Graph screenshot, IDA if-else graph screenshot, and IDA loop graph screenshot.

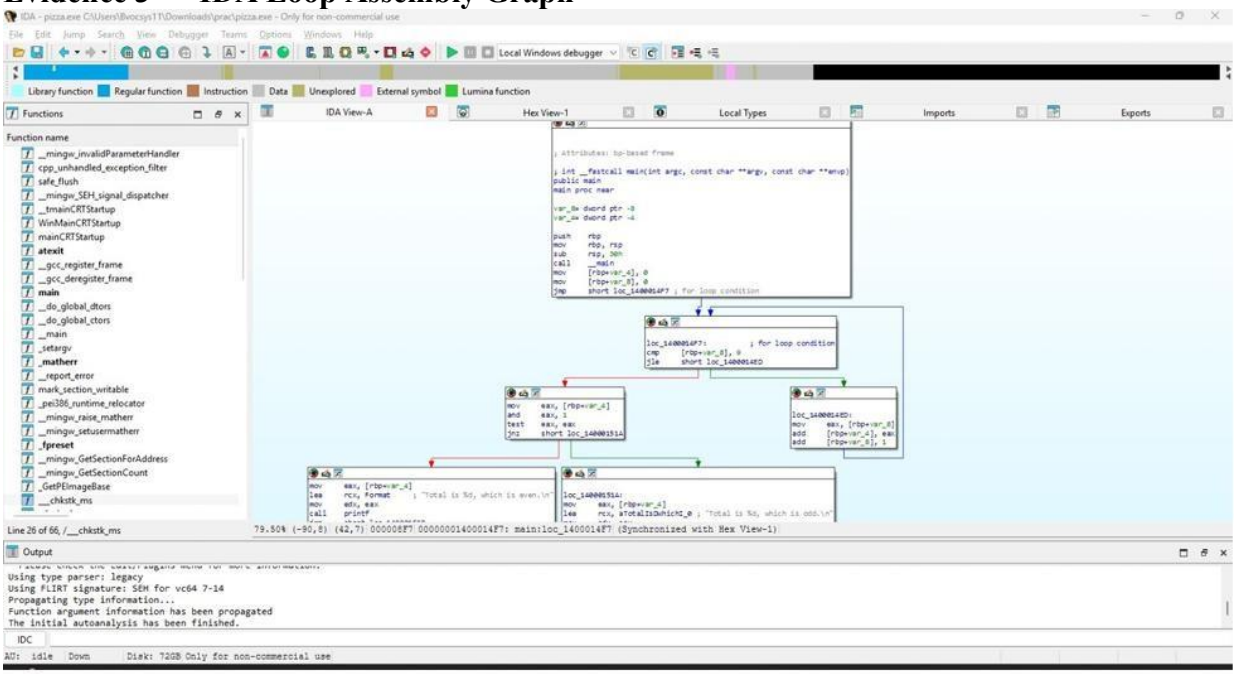
Evidence 1 — IDA Xref Graph



Evidence 2 — IDA If-Else Assembly Graph



Evidence 3 — IDA Loop Assembly Graph



PRACTICAL 5

Spot Windows Tools (APIs) Being Used

Windows API Recognition and Static Analysis using IDA Free

1. Aim

To find which built-in Windows tools (APIs) a program uses, and to use those APIs as clues to understand the program's purpose.

2. Tools and Environment

Item	Version / Detail
Operating system	Not provided in the supplied screenshots.
Compiler	MinGW-w64 / GCC (version not provided).
IDA Free	Version not visible in the supplied screenshots.
Virtual machine used?	Not provided. The guide recommends using a VM if the compiled program is executed.

3. Theory

What is a Windows API? A Windows API is a function provided by Windows that lets a program request an operating-system operation, such as opening a file, changing the registry, or starting another program.

Why do malware analysts study imported APIs? Imported APIs give clues about what an unknown program can do. Analysts can often infer file, registry, process, or other activity without running the program.

What are Nt / Zw functions? Nt and Zw functions are lower-level Windows interfaces. A program may use them to access operating-system services more directly.

What do the A and W suffixes mean? The A suffix denotes an ANSI version of a function, while W denotes the wide/Unicode version.

4. Procedure

1. Prepared the C test program containing CreateFileA, RegCreateKeyExA / RegSetValueExA and WinExec.
2. Compiled the source with GCC to produce the Windows executable.
3. Opened the compiled executable in IDA Free and waited for auto-analysis to finish.
4. Opened the Imports view and confirmed the relevant functions under ADVAPI32 and KERNEL32.
5. Located the API calls and examined the instructions immediately before each call to identify arguments.
6. Read the referenced strings such as the file path, registry subkey, registry value data and command line.
7. Matched the numeric values and arguments with their Windows API meanings.
8. Used the API combination to determine the likely purpose of the harmless test program.

5. Source Code

```
#include <windows.h>
#include <string.h>

int main(void)
{
    HANDLE hFile = CreateFileA("C:\\Temp\\practical6_test.txt",
                    GENERIC_WRITE, 0, NULL,
                    CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hFile != INVALID_HANDLE_VALUE) CloseHandle(hFile);

    HKEY hKey;
    LONG rc = RegCreateKeyExA(HKEY_CURRENT_USER,
                    "Software\\Practical6Test",
                    0, NULL, 0, KEY_SET_VALUE, NULL,
                    &hKey, NULL);
    if (rc == ERROR_SUCCESS)
    {
        const char *data = "HelloLab";
        RegSetValueExA(hKey, "TestValue", 0, REG_SZ,
                       (const BYTE *)data, (DWORD)strlen(data) + 1);
        RegCloseKey(hKey);
    }

    WinExec("notepad.exe", SW_SHOWNORMAL);
    return 0;
}
```

6. Observations

6.1 Imports Confirmed in IDA

API	DLL	Version
CreateFileA	KERNEL32.dll	A – ANSI
RegCreateKeyExA	ADVAPI32.dll	A – ANSI
RegSetValueExA	ADVAPI32.dll	A – ANSI
WinExec	KERNEL32.dll	No A/W suffix

6.2 Arguments Found in IDA

API	Arguments seen in IDA
CreateFileA	**Path:** “C:\\Temp\\practical6_test.txt” **Access:** 40000000h **Disposition:** 2
RegCreateKeyExA	**Root key:** 0FFFFFFFF80000001h **Subkey:** “Software\\Practical6Test” **Access:** 2
RegSetValueExA	**Value name:** “TestValue” **Type:** 1 **Data:** “HelloLab” **Size:** 9
WinExec	**Command:** “notepad.exe” **Show flag:** 1

7. Tool Matched to Official Purpose

API	Official purpose	What it did in this program	Target found in IDA
CreateFileA	Creates or opens a file or I/O device.	Creates/overwrites a text file for writing.	C:\Temp\practical6_test.txt
RegCreateKeyExA	Creates a registry key or opens it if it already exists.	Creates/opens the test registry key under the current user.	HKCU\Software\Practical6Test
RegSetValueExA	Sets the data and type of a registry value.	Writes a text registry value.	TestValue = HelloLab
WinExec	Runs a specified application from a command line; it is a legacy API.	Launches Notepad.	notepad.exe

8. Purpose of the Program

Based on the imported APIs and the arguments visible in IDA, this program creates or overwrites a text file at C:\Temp\practical6_test.txt, creates or opens a registry key under the current user's registry hive, writes the string HelloLab as TestValue, and then starts notepad.exe. The combination resembles file creation, registry modification and process launching, which are useful behavioral clues during malware analysis. In this particular practical, the targets are harmless and the program is a benign test program.

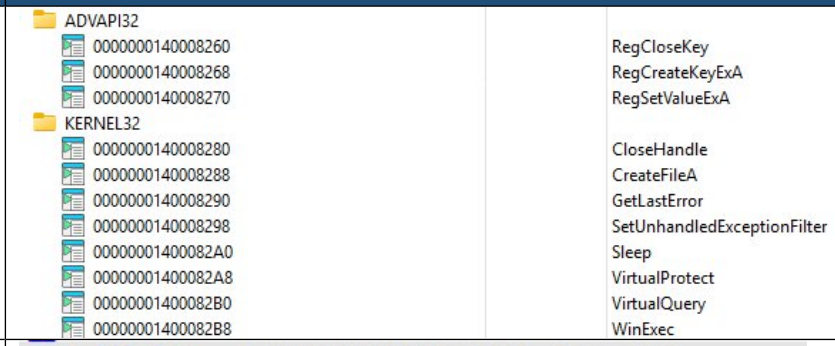
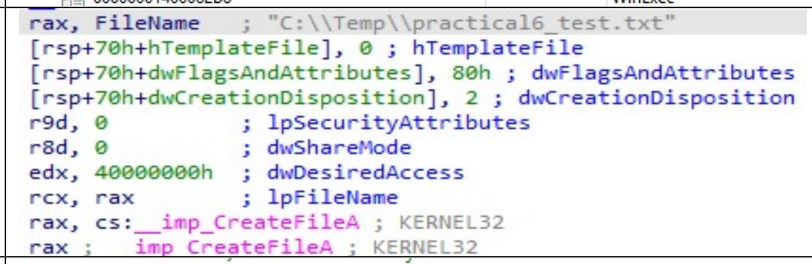
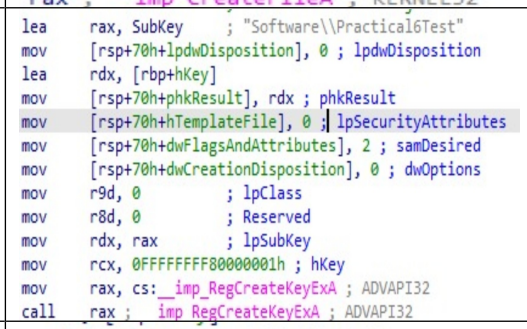
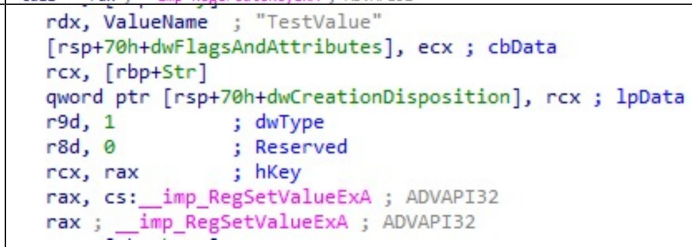
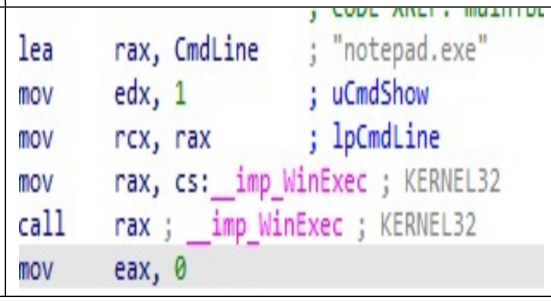
9. Conclusion

Based on the imported APIs, this program creates a file on disk, stores a setting in the current user's registry, and then starts another program. This pattern of file, registry and process activity is typical of a program that drops a file, configures itself, and launches a payload. In this case the targets are harmless (a text file, a private registry key and Notepad), so it is a benign test program.

10. Precautions Followed

- The analysis was performed by viewing the compiled program in IDA rather than relying on execution.
- Only harmless targets were used: a text file, a private registry key and Notepad.
- The API evidence was interpreted from the supplied IDA screenshots and the Practical 6 guide.

11. Figures List

Figure	Caption
IDA Imports tab showing ADVAPI32 and KERNEL32	
CreateFileA call with arguments	
RegCreateKeyExA call with arguments	
RegSetValueExA call with arguments	
WinExec call with arguments	

Malware Analysis Case Studies

WannaCry Ransomware and Emotet Banking Trojan

Case Study 1: WannaCry Ransomware (2017)

Background

WannaCry was a ransomware worm that spread across the world in May 2017, infecting over 200,000 systems in about 150 countries. It hit organizations such as the UK's NHS, causing serious disruption. It is widely attributed to the Lazarus Group.

Sample Overview

Attribute	Detail
Type	Ransomware with worm capability
File format	Windows PE (32-bit EXE) dropper
Propagation	EternalBlue exploit for SMBv1 (MS17-010), plus the DoublePulsar backdoor
Impact	Encrypted user files (renamed with a .WNCRY extension) and demanded a Bitcoin ransom

Static Analysis Findings

1. **File identification:** The sample is a PE file whose hashes (MD5/SHA-256) were widely published, so VirusTotal lookups show high detection.
2. **Strings:** The dropper contains a hard-coded kill-switch domain (a long random-looking URL), the service name mssecsvc2.0, ransom note text and Bitcoin wallet addresses.
3. **PE structure:** The .rsrc section is unusually large because it hides an embedded payload (the ransomware component, tasksche.exe), which the dropper extracts at runtime.
4. **Imports (APIs):** The imported APIs are summarized in the table below.

API	What it suggests
InternetOpenA, InternetOpenUrlA	Network connection (kill-switch check)
FindResourceA, LoadResource, LockResource	Extracting the embedded payload from resources
CreateServiceA, OpenSCManagerA	Installing itself as a Windows service (persistence)
CreateFileA, WriteFile, CreateProcessA	Dropping and launching the payload

5. **Disassembly (IDA Free):** The entry logic shows the dropper trying to open the kill-switch URL. If the connection succeeds, it exits. If it fails, it continues to the infection routine. This explains how researcher Marcus Hutchins stopped the outbreak by registering the domain.

Key Lessons

- Embedded resources are a common way to hide a second-stage payload.
- Static analysis can find a kill switch without executing the sample.
- The import table gives a fast behavioral summary (network, service creation, resource extraction).
- Patching known vulnerabilities (MS17-010) would have prevented the spread.

Mapping to practicals: Static analysis (2), PE structure and .rsrc section (3), assembly reading of the kill-switch logic (4), API identification (5).

Case Study 2: Emotet Banking Trojan and Loader (2014–2021)

Background

Emotet began as a banking trojan in 2014 and evolved into one of the most dangerous malware-as-a-service botnets. Infected machines were rented out to other criminals to deliver payloads such as TrickBot, QakBot and Ryuk ransomware. Europol and partner agencies took down its infrastructure in January 2021, though it reappeared in late 2021.

Sample Overview

Attribute	Detail
Type	Trojan, botnet and malware loader
Delivery	Phishing emails with malicious Word/Excel attachments containing macros
File formats	Office document (OLE/OOXML) as the first stage, then a PE DLL/EXE as the second stage
Persistence	Registry Run keys or Windows services

Static Analysis Findings

1. **Stage 1, the document:** Tools such as olevba or oledump reveal heavily obfuscated VBA macros that build and run a PowerShell command, often Base64-encoded, to download the next stage.
2. **Stage 2, the PE file:**
   - High-entropy sections indicate packing or encryption.
   - A sparse import table, with mostly LoadLibrary and GetProcAddress, shows that APIs are resolved dynamically at runtime to evade static detection.
   - Meaningful strings are scarce because most are encrypted.
3. **Suspicious APIs:** Once resolved, the suspicious APIs are summarized in the table below.

API	What it suggests
VirtualAlloc, VirtualProtect, WriteProcessMemory	Unpacking and injecting code in memory
InternetConnect, HttpSendRequest, WinHttp*	Communication with command-and-control (C2) servers
RegSetValueEx, CreateService	Persistence
CryptEncrypt, CryptDecrypt	Encrypted C2 traffic or configuration

4. **Disassembly (IDA Free):** Loops with XOR or similar routines indicate string and configuration decryption, and the list of C2 IP addresses is stored encrypted inside the binary.

Key Lessons

- Modern malware is multi-stage, so analysts must examine both the document and the executable.

- A small import table combined with high entropy is a strong sign of packing.
- Static analysis alone may not reveal everything, so it is often combined with dynamic analysis (sandbox) and memory analysis.
- Defenses include disabling Office macros, email filtering and user awareness training.

Mapping to practicals: Static analysis (2), PE structure and entropy (3), decryption routines in assembly (4), dynamic API resolution (5).

Comparison

Feature	WannaCry	Emotet
Category	Ransomware worm	Trojan, loader, botnet
Infection vector	SMB vulnerability (EternalBlue)	Phishing with macro documents
Main goal	Extortion	Data theft and delivering other malware
Static clues	Large .rsrc, kill-switch URL, service APIs	Obfuscated macros, packed PE, dynamic API resolution

Safety note: Analyze real samples only inside an isolated virtual machine with networking disabled or host-only, and take a snapshot before starting.